

Auditoría del proyecto

Qué falta, qué está roto y en qué orden atacarlo.

Contrastado contra la especificación funcional del primer parcial.

Resumen

El proyecto tiene el **esqueleto completo y funcionando**: navegación entre las 10 pantallas, sistema de usuarios, tablero interactivo con assets, motor de turnos con ruleta e historial. Lo que falta se concentra en dos frentes: las **habilidades especiales** (ninguna implementada) y los **requisitos técnicos** del enunciado (tres sin cumplir, y son los que penalizan).

ÁREA	ESTADO	DETALLE
Sistema de usuarios y menús	Completo	Login, registro, mi cuenta, navegación
Tablero e interfaz de partida	Completo	Casillas-botón, assets, resaltado, historial
Movimiento y ataque normal	Casi	Falta validar el camino del Lobo
Habilidades especiales	0 de 5	Absorción, lanza, invocar, atacar vía Zombie, elegir ataque
Zombie	Inalcanzable	La clase existe pero nunca llega al tablero
Reglas de ruleta	Simplificada	Falta la regla de giros extra
Reportes	Con fallos	Ranking mal formado; contadores en cero
Requisitos técnicos	5 de 8	Ver Parte C

LOS TRES BLOQUEANTES

1. `Ficha` no es abstracta y las subclases no sobrescriben nada. Esto bloquea a la vez el requisito 1, el 2 y todas las habilidades especiales.
2. No existe ninguna `interface` en el proyecto y los datos viven en `ArrayList`. El requisito 5 pide arreglos accedidos a través de una interfaz.
3. Solo hay una función recursiva. El requisito 4 pide dos distintas.

Parte A · Bugs

Severidad: **ALTA** rompe una regla del enunciado o el programa **MEDIA** comportamiento incorrecto visible **BAJA** calidad y robustez

A1 · La Muerte tiene vida y ataque invertidos **ALTA**

TipoFicha.java línea 8

El constructor es `TipoFicha(nombre, vida, ataque, escudo, movilidad)`. La llamada actual es `new TipoFicha("Muerte", 4, 3, 1, ...)`, o sea **vida 4 y ataque 3**. Según la Figura 2 del enunciado la Muerte tiene **ataque 4 y vida 3**. Están al revés.

PIEZA	ENUNCIADO (ATQ/VIDA/ESC)	CÓDIGO (ATQ/VIDA/ESC)	
Hombre Lobo	5 / 5 / 2	5 / 5 / 2	ok
Vampiro	3 / 4 / 5	3 / 4 / 5	ok
Muerte	4 / 3 / 1	3 / 4 / 1	invertido
Zombie	1 / 1 / 0	1 / 1 / 0	ok

Consecuencia: la Muerte pega 3 en vez de 4, y su lanza debería hacer 2 (la mitad de 4).

A2 · Los contadores de partidas nunca se incrementan **ALTA**

Usuario.java · Juego.terminar()

`Usuario.ganadas` y `Usuario.perdidas` se inicializan a 0 y **no hay ni un solo punto del código que los aumente**. No existe setter ni método de incremento. `PanelMiInformacion` los muestra, así que siempre se verá «Partidas Ganadas: 0».

A3 · El ranking inventa filas y no tiene las columnas pedidas **ALTA**

PanelRanking.java línea 34 · GestorUsuarios.getUsuariosOrdenados()

Dos problemas juntos:

- `numJugadores = 10` está fijo. Con 3 jugadores registrados se pintan **7 filas con el texto «-- jugador null --»**. El enunciado pide «el listado de **todos** los jugadores activos».
- El enunciado pide tres columnas: **Posición, Usuario, Puntos**. El código concatena nombre y puntos en un solo string ("`nombre - N puntos`") y usa una rejilla de 2 columnas.

A4 · Ganar por eliminación no cierra la partida **ALTA**

PanelJuego.actualizarCabecera()

Al retirarse sí se vuelve al menú (línea 116), pero cuando se gana **destruyendo todas las piezas** la interfaz solo escribe «Partida terminada» y se queda ahí. El enunciado es explícito: «tras lo cual **se cierra la ventana de la partida y se regresa exclusivamente al Menú Principal**».

A5 · Retirarse no pide confirmación **ALTA**

PanelJuego.java línea 114

El botón llama a `juego.retirarse()` directamente. El enunciado: «Dicha acción **debe solicitar confirmación** antes de ejecutarse; una vez confirmada, el jugador contrario es declarado ganador automáticamente». Un clic accidental pierde la partida.

A6 · El Hombre Lobo salta por encima de las fichas **MEDIA**

Ficha.dentroRango() · Juego.esDestinoValido()

`movilidad` incluye los desplazamientos de 2 casillas, pero nadie comprueba la casilla intermedia. El Lobo puede saltar sobre una pieza propia o enemiga. El enunciado dice «hasta 2 casillas **vacías**»: el plural implica que el camino debe estar libre.

A7 • Los mensajes de error nunca se ven **MEDIA**

PanelJuego.alPulsarCasilla() • actualizarCabecera()

avisar(...) escribe en labelMensaje, y acto seguido redibujar() llama a actualizarCabecera(), que **sobrescribe esa misma etiqueta** con el texto genérico. El aviso se pierde siempre. Solución: que actualizarCabecera lea de juego.getUltimoMensaje().

A8 • requireNonNull puede tumbar el programa **MEDIA**

PanelSelector.java línea 39

Objects.requireNonNull(comboBox.getSelectedItem()) lanza NullPointerException si el combo está vacío. Hoy lo protege el chequeo de PanelMenuPrincipal, pero es una excepción no controlada esperando ocurrir, y el **requisito 6** prohíbe terminar abruptamente.

A9 • Varios paneles asumen que hay sesión activa **MEDIA**

PanelMiInformacion • PanelHistorial

Ambos hacen gestorUsuarios.getUsuarioActual().getX() sin comprobar null. Si se llega a esas pantallas sin sesión, NullPointerException. Mismo riesgo de requisito 6.

A10 • Se navega antes de cerrar sesión **BAJA**

PanelMenuPrincipal línea 40 • PanelMiCuenta línea 32

El orden es mostrarPanel(new PanelInicio()) y **después** cerrarSesion() / cerrarCuenta(). Funciona porque PanelInicio no consulta al usuario, pero es frágil: si algún día lo hace, leerá una sesión que ya debería estar cerrada. Invertir las dos líneas.

A11 • Usuario.getHistorial() expone la lista real **BAJA**

Usuario.java

Devuelve el ArrayList interno, así que cualquiera puede modificarlo desde fuera. Se arregla al pasar a arreglos (requisito 5), que además resuelve el problema de fondo.

A12 • La contraseña se muestra en texto plano **BAJA**

PanelMiInformacion.java línea 37

«Ver mi información» imprime la clave tal cual. El enunciado no lo prohíbe, pero es un detalle que un evaluador puede comentar. Además los campos de contraseña son JTextField y no JPasswordField.

Parte B · Mecánicas que faltan

Todas salen directamente del enunciado. Ninguna está implementada.

#	MECÁNICA	REGLA EXACTA	DÓNDE VA
B1	Absorción de sangre Vampiro	En lugar del ataque normal, sobre enemigo adyacente. Resta 1 punto (no 3) y el Vampiro recupera 1 de vida .	FichaVampiro
B2	Lanza a distancia Necrómante	Enemigo a 2 casillas . Ignora el escudo . Daño 2 . Ojo: el enunciado se contradice sobre la dirección (ver nota abajo).	FichaMuerte
B3	Invocar Zombie Necrómante	En cualquier casilla vacía del tablero, sin límite de distancia. El Necrómante no se mueve.	FichaMuerte + Juego
B4	Atacar vía Zombie Necrómante	Enemigo que no esté ni a 1 ni a 2 casillas del Necrómante, y que esté adyacente a un Zombie propio. Daño 1 .	Juego
B5	Elegir tipo de ataque	Al atacar a una pieza enemiga, «se ofrece al jugador la posibilidad de elegir entre un ataque normal o un ataque especial».	PanelJuego + Juego
B6	Elegir mover o invocar	Con el Necrómante y destino vacío, «el sistema debe ofrecer al jugador la posibilidad de elegir entre ambas opciones».	PanelJuego
B7	Giros extra	2 piezas perdidas → hasta 2 giros. 4 perdidas → hasta 3. Agotados todos, pierde el turno.	Juego.girarRuletaValida()
B8	Camino libre del Lobo	«hasta 2 casillas vacías »: la intermedia también cuenta.	Juego.esDestinoValido()

ESTADO DEL ZOMBIE

La clase `FichaZombie` existe y está bien configurada (1/1/0, movilidad vacía), pero **nunca llega al tablero** porque no hay forma de invocarlo. Hasta que B3 esté hecho, el Zombie es código muerto. Ojo también: la condición de victoria debe contar los Zombies invocados — `tieneFichas()` ya lo hace correctamente.

CONTRADICCIÓN DEL ENUNCIADO SOBRE LA LANZA

La sección 3.3 dice «a 2 casillas de distancia, **en cualquier dirección**». La sección 7.3 dice «a 2 casillas de distancia, **en línea horizontal o vertical y sin obstrucciones**». Son incompatibles. Recomendación: implementar la versión de 7.3 (la más restrictiva, y está en la sección de implementación) y dejarlo comentado en el código para poder justificarlo.

Parte C · Requisitos técnicos

El enunciado avisa: «El cumplimiento de estos requisitos **no otorga puntos adicionales**, pero su **incumplimiento sí implica penalizaciones**». Cinco de ocho están resueltos.

#	REQUISITO	ESTADO	SITUACIÓN REAL
1	Herencia y polimorfismo en las piezas	Parcial	Hay herencia (<code>FichaLobo extends Ficha</code>) pero cero polimorfismo : las cuatro subclases solo llaman a <code>super()</code> y no sobrescriben ni un método. Verificado: 0 <code>@Override</code> en todo el paquete <code>fichas</code> .
2	Clase abstracta con función abstracta	Al límite	<code>PanelAbstracto</code> es abstracta y tiene <code>iniciarPanel()</code> abstracto, así que la letra del requisito se cumple. Pero el requisito 1 pide que sea la lógica de las piezas la que use herencia y polimorfismo, y ahí <code>Ficha</code> es una clase normal. Conviene reforzarlo.
3	Clase final y funciones finales	A medias	Clases final: <code>TipoFicha</code> y <code>CargadorAssets</code> . Funciones finales: ninguna . El requisito pide las dos cosas («una clase final, así como funciones finales »).
4	Dos funciones recursivas distintas	Falta	Solo hay una : <code>GestorUsuarios.ordenarRecursivo()</code> . <code>particion()</code> no es recursiva. Los pares <code>dentroRango</code> , <code>agregarBoton</code> , <code>agregarPanel</code> y <code>getGBC</code> son sobrecargas , no recursión.
5	Arreglos + interfaz de almacenamiento	No cumple	Verificado: no existe ninguna interface en el proyecto . Y los datos usan <code>ArrayList</code> : <code>GestorUsuarios.usuarios</code> , <code>Usuario.historial</code> , <code>Juego.historial</code> . El requisito pide arreglos y una interfaz que encapsule el acceso. Es el requisito más caro de los que faltan.
6	Nunca terminar por excepción no controlada	En riesgo	No hay caídas conocidas hoy, pero quedan tres puntos frágiles: A8, A9 y cualquier <code>getUsuarioActual()</code> sin comprobar. Conviene un <code>try/catch</code> de último recurso en <code>Launcher</code> .
7	<code>main</code> sin lógica del juego	Cumple	<code>Main</code> solo instancia <code>Launcher</code> . Impecable.
8	Interfaz gráfica, no consola	Cumple	Swing en todo el flujo. El motor ya no imprime nada: registra en el <code>historial</code> .

LA BUENA NOTICIA

Los requisitos 1, 2, 3 y 4 se resuelven casi todos con **el mismo trabajo**: convertir `Ficha` en abstracta con un método abstracto por habilidad. Eso da herencia real, polimorfismo, la clase abstracta en el sitio correcto, y un lugar natural para poner métodos `final`. Por eso es la Fase 1 del plan.

Parte D · Orden de trabajo

Ordenado para que cada fase desbloquee la siguiente, y para que lo que más penaliza esté hecho pronto. Las fases 0 y 1 son las que más rinden por tiempo invertido.

FASE 0 · ARREGLOS DE MINUTOS

Quitar la sangre antes de operar

Cambios de una o dos líneas, sin riesgo. Hazlos ya para no arrastrarlos.

- **A1** — invertir vida y ataque de la Muerte en `TipoFicha`.
- **A7** — que `actualizarCabecera` use `juego.getUltimoMensaje()`.
- **A10** — invertir el orden de navegación y cierre de sesión.
- **A5** — envolver el botón Retirarse en un `JOptionPane.showConfirmDialog`.
- **A8 / A9** — comprobar `null` antes de usar el usuario activo y el combo.

FASE 1 · LA REFACTORIZACIÓN CLAVE

Ficha abstracta y polimórfica

Resuelve requisitos 1, 2 y 3, y abre la puerta a todas las habilidades. Es el cuello de botella de todo lo demás.

- Hacer `Ficha` `abstract` con al menos un método abstracto, por ejemplo `ataqueEspecial(Celda origen, Celda destino, Tablero t)` y `puedeUsarEspecial(...)`.
- Implementarlos distinto en `FichaVampiro`, `FichaLobo`, `FichaMuerte` y `FichaZombie`.
- Marcar como `final` los métodos que no deben cambiar en las hijas: `recibirAtaque`, `dentroRango`, los getters.
- Sustituir en `Juego.ejecutar()` los `if` por llamadas polimórficas.

FASE 2 · REGLAS DEL JUEGO

Las cinco habilidades y las dos elecciones

- **B8** primero: camino libre del Lobo. Es el más simple y valida la nueva estructura.
- **B1** absorción de sangre del Vampiro.
- **B5** diálogo de ataque normal contra especial en `PanelJuego`.
- **B2** lanza del Necrómante (decidir la contradicción y documentarla).
- **B3** y **B6** invocar Zombie y el diálogo de mover-o-invocar. Con esto el Zombie por fin aparece en el tablero.
- **B4** ataque a través de Zombie, que depende de B3.
- **B7** giros extra de la ruleta.

FASE 3 · REQUISITO 5

Arreglos detrás de una interfaz

El requisito más caro. Hazlo cuando la lógica ya esté estable, para tocar el almacenamiento una sola vez.

- Crear una `interface` de almacenamiento, por ejemplo `Almacen<T>` con `agregar`, `obtener`, `buscar`, `tamano`, `aArreglo`.
- Implementarla con un arreglo redimensionable a mano (crear uno mayor y copiar al llenarse).
- Migrar `GestorUsuarios.usuarios`, `Usuario.historial` y `Juego.historial` a esa interfaz.
- Verificar que el enunciado se cumple en su totalidad: arreglos **y** interfaz.

FASE 4 · REQUISITO 4

La segunda función recursiva

Ya tienes `ordenarRecursivo`. Falta una más, y debe ser **distinta**.

- Candidata natural: **validar el camino libre** del Lobo o de la lanza avanzando casilla a casilla (`caminoLibre(origen, destino, paso)` que se llama a sí misma hasta llegar). Encaja con B8 y B2, así que sale casi gratis.
- Otra opción: recorrer el tablero buscando destinos alcanzables de forma recursiva.
- Evita confundir sobrecarga con recursión: el método debe llamarse **a sí mismo**.

FASE 5 · CIERRE Y REPORTES

Que la partida termine bien y los datos cuadren

- **A4** — al ganar por eliminación, cerrar la partida y volver al Menú Principal.
- **A2** — incrementar `ganadas` y `perdidas` en `Juego.terminar()`. Hace falta añadir los métodos en `Usuario`.
- **A3** — ranking con todos los activos y tres columnas reales: Posición, Usuario, Puntos.
- Comprobar que el historial del jugador muestra los mensajes literales del enunciado, del más reciente al más antiguo.

FASE 6 · PULIDO

Lo que suma si sobra tiempo

- **A12** — `JPasswordField` y ocultar la clave en «Mi información».
- Mostrar vida y escudo sobre cada ficha del tablero, o en un panel de info al seleccionar.
- Llenar el `inferiorPanel` con las piezas comidas.
- Un recurso visual mejor para la ruleta que una etiqueta de texto.
- `try/catch` global en `Launcher` como red de seguridad del requisito 6.

Comprobación final antes de entregar

VERIFICAR	CÓMO
Los 4 mensajes literales del enunciado	Ataque, destrucción, victoria y retiro, palabra por palabra
Contraseña de exactamente 5 caracteres	Probar con 4, 5 y 6
Nombre de usuario único	Registrar dos veces el mismo
Sin oponente no se puede jugar	Con un solo usuario registrado
Ranking solo con activos	Cerrar una cuenta y volver a mirar
Nada de archivos en disco	Buscar <code>FileWriter</code> , <code>FileOutputStream</code> , <code>Files</code> .
El programa no se cae nunca	Clics rápidos, campos vacíos, navegar sin sesión
Los 8 requisitos técnicos	Recorrer la tabla de la Parte C uno por uno